

Работа с PyQt, текстовыми файлами (.txt) и JSON-файлами

Что такое JSON?

JSON (JavaScript Object Notation) — это легкий формат обмена данными, который легко читается и записывается как людьми, так и компьютерами.

Основные особенности JSON:

- Текстовый формат, независимый от языка программирования
- Использует пары "ключ-значение"
- Поддерживает следующие типы данных: строки, числа, логические значения, массивы, списки, объекты, null
- Имеет простой и понятный синтаксис

Пример JSON-объекта:

json

```
{
  "name": "Тулькубаев Ильнар",
  "age": 20,
  "is_student": true,
  "courses": ["ОАиП", "РМП"],
  "address": {
    "street": "123 Что-то там",
    "city": "Уфа"
  }
}
```

Настройка окружения

Перед началом работы установите PyQt5:

Настройка окружения

```
pip install PyQt5
```

1. Работа с текстовыми файлами (.txt) в PyQt

1.1 Чтение из текстового файла

Чтение из текстового файла

```
import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow, QTextEdit,
                             QPushButton, QVBoxLayout, QWidget, QFileDialog)

class TextFileReader(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        self.setWindowTitle('Текстовый редактор')
        self.setGeometry(100, 100, 600, 400)

        self.text_edit = QTextEdit()
        self.open_btn = QPushButton('Открыть файл')
        self.open_btn.clicked.connect(self.open_file)

        layout = QVBoxLayout()
        layout.addWidget(self.text_edit)
        layout.addWidget(self.open_btn)

        container = QWidget()
        container.setLayout(layout)
        self.setCentralWidget(container)

    def open_file(self):
        file_name, _ = QFileDialog.getOpenFileName(self, 'Открыть файл', '', 'Текстовые файлы (*.txt)')
        if file_name:
            with open(file_name, 'r', encoding='utf-8') as file:
                content = file.read()
                self.text_edit.setPlainText(content)

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = TextFileReader()
    window.show()
    sys.exit(app.exec_())
```

1.2 Запись в текстовый файл

Запись в текстовый файл

```
import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow, QTextEdit,
                             QPushButton, QVBoxLayout, QWidget, QFileDialog)

class TextFileWriter(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        self.setWindowTitle('Текстовый редактор')
        self.setGeometry(100, 100, 600, 400)

        self.text_edit = QTextEdit()
        self.save_btn = QPushButton('Сохранить файл')
        self.save_btn.clicked.connect(self.save_file)

        layout = QVBoxLayout()
        layout.addWidget(self.text_edit)
        layout.addWidget(self.save_btn)

        container = QWidget()
        container.setLayout(layout)
        self.setCentralWidget(container)

    def save_file(self):
        file_name, _ = QFileDialog.getSaveFileName(self, 'Сохранить файл', '', 'Текстовые файлы (*.txt)')
        if file_name:
            with open(file_name, 'w', encoding='utf-8') as file:
                file.write(self.text_edit.toPlainText())

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = TextFileWriter()
    window.show()
    sys.exit(app.exec_())
```

1.3 Редактирование и удаление данных в текстовом файле

Редактирование и удаление данных в текстовом файле

```
import sys
import os
from PyQt5.QtWidgets import (QApplication, QMainWindow, QTextEdit, QPushButton,
                             QVBoxLayout, QWidget, QFileDialog, QMessageBox)

class TextFileEditor(QMainWindow):
    def __init__(self):
        super().__init__()
        self.current_file = None
        self.initUI()

    def initUI(self):
        self.setWindowTitle('Редактор текстовых файлов')
        self.setGeometry(100, 100, 600, 400)

        self.text_edit = QTextEdit()

        self.open_btn = QPushButton('Открыть')
        self.open_btn.clicked.connect(self.open_file)

        self.save_btn = QPushButton('Сохранить')
        self.save_btn.clicked.connect(self.save_file)

        self.save_as_btn = QPushButton('Сохранить как')
        self.save_as_btn.clicked.connect(self.save_file_as)

        self.clear_btn = QPushButton('Очистить')
        self.clear_btn.clicked.connect(self.clear_text)

        self.delete_btn = QPushButton('Удалить файл')
        self.delete_btn.clicked.connect(self.delete_file)

        layout = QVBoxLayout()
        layout.addWidget(self.text_edit)
        layout.addWidget(self.open_btn)
        layout.addWidget(self.save_btn)
        layout.addWidget(self.save_as_btn)
        layout.addWidget(self.clear_btn)
        layout.addWidget(self.delete_btn)

        container = QWidget()
        container.setLayout(layout)
        self.setCentralWidget(container)

    def open_file(self):
        file_name, _ = QFileDialog.getOpenFileName(self, 'Открыть файл', '', 'Текстовые файлы (*.txt)')
        if file_name:
            self.current_file = file_name
            with open(file_name, 'r', encoding='utf-8') as file:
                content = file.read()
                self.text_edit.setPlainText(content)
```

```
def save_file(self):
    if self.current_file:
        with open(self.current_file, 'w', encoding='utf-8') as file:
            file.write(self.text_edit.toPlainText())
        QMessageBox.information(self, 'Успех', 'Файл успешно сохранен!')
    else:
        self.save_file_as()

def save_file_as(self):
    file_name, _ = QFileDialog.getSaveFileName(self, 'Сохранить файл', '', 'Текстовые файлы (*.txt)')
    if file_name:
        self.current_file = file_name
        with open(file_name, 'w', encoding='utf-8') as file:
            file.write(self.text_edit.toPlainText())
        QMessageBox.information(self, 'Успех', 'Файл успешно сохранен!')

def clear_text(self):
    self.text_edit.clear()

def delete_file(self):
    if self.current_file:
        reply = QMessageBox.question(self, 'Подтверждение',
                                     f'Вы уверены, что хотите удалить файл {self.current_file}?',
                                     QMessageBox.Yes | QMessageBox.No, QMessageBox.No)
        if reply == QMessageBox.Yes:
            try:
                os.remove(self.current_file)
                self.current_file = None
                self.text_edit.clear()
                QMessageBox.information(self, 'Успех', 'Файл успешно удален!')
            except Exception as e:
                QMessageBox.critical(self, 'Ошибка', f'Не удалось удалить файл: {str(e)}')
        else:
            QMessageBox.warning(self, 'Предупреждение', 'Нет открытого файла для удаления')

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = TextFileEditor()
    window.show()
    sys.exit(app.exec_())
```

2. Работа с JSON-файлами в PyQt

2.1 Запись и чтение с JSON-файл

Запись и чтение с JSON-файл

```
import json
import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
                             QHBoxLayout, QLabel, QLineEdit, QPushButton,
                             QTextEdit, QFileDialog, QMessageBox)

class JSONEditor(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("JSON Editor")
        self.setGeometry(100, 100, 600, 400)

        # Основной виджет и layout
        self.central_widget = QWidget()
        self.setCentralWidget(self.central_widget)
        self.main_layout = QVBoxLayout()
        self.central_widget.setLayout(self.main_layout)

        # Виджеты для ввода данных
        self.create_input_widgets()

        # Кнопки действий
        self.create_action_buttons()

        # Поле для вывода JSON
        self.output_text = QTextEdit()
        self.output_text.setReadOnly(True)
        self.main_layout.addWidget(self.output_text)

        # Текущий файл
        self.current_file = None

    def create_input_widgets(self):
        # Создаем виджеты для ввода данных
        input_group = QWidget()
        input_layout = QVBoxLayout()
        input_group.setLayout(input_layout)

        # Поля для ключа и значения
        self.key_input = QLineEdit()
        self.key_input.setPlaceholderText("Введите ключ")
        self.value_input = QLineEdit()
        self.value_input.setPlaceholderText("Введите значение")

        # Кнопка добавления пары
        self.add_button = QPushButton("Добавить пару")
        self.add_button.clicked.connect(self.add_pair)

        input_layout.addWidget(QLabel("Ключ:"))
        input_layout.addWidget(self.key_input)
        input_layout.addWidget(QLabel("Значение:"))
        input_layout.addWidget(self.value_input)
        input_layout.addWidget(self.add_button)

        self.main_layout.addWidget(input_group)
```

```
def create_action_buttons(self):
    """Создаем кнопки действий"""
    button_group = QWidget()
    button_layout = QHBoxLayout()
    button_group.setLayout(button_layout)

    # Кнопки
    self.create_button = QPushButton("Создать JSON")
    self.create_button.clicked.connect(self.create_json)

    self.save_button = QPushButton("Сохранить в файл")
    self.save_button.clicked.connect(self.save_to_file)

    self.load_button = QPushButton("Загрузить из файла")
    self.load_button.clicked.connect(self.load_from_file)

    self.clear_button = QPushButton("Очистить")
    self.clear_button.clicked.connect(self.clear_all)

    button_layout.addWidget(self.create_button)
    button_layout.addWidget(self.save_button)
    button_layout.addWidget(self.load_button)
    button_layout.addWidget(self.clear_button)

    self.main_layout.addWidget(button_group)
```

```
def add_pair(self):
    """Добавление новой пары ключ-значение в JSON-структуру"""

    # Получаем текст из поля ввода ключа, удаляя пробелы по краям
    key = self.key_input.text().strip()

    # Получаем текст из поля ввода значения, удаляя пробелы по краям
    value = self.value_input.text().strip()

    # Проверяем, что ключ не пустой
    if not key:
        # Показываем предупреждение, если ключ пустой
        QMessageBox.warning(self, "Ошибка", "Ключ не может быть пустым!")
        # Выходим из функции, не выполняя дальнейшие действия
        return

    # Проверяем, состоит ли значение только из цифр (целое число)
    if value.isdigit():
        # Преобразуем строку в целое число
        value = int(value)
    # Проверяем, является ли значение числом с плавающей точкой
    elif value.replace('.', '', 1).isdigit() and value.count('.') < 2:
        # Преобразуем строку в число с плавающей точкой
        value = float(value)

    # Проверяем, пусто ли поле вывода JSON
    if not self.output_text.toPlainText():
        # Создаем новый пустой словарь, если поле пустое
        data = {}
    else:
        # Если поле не пустое, загружаем существующий JSON как словарь
        data = json.loads(self.output_text.toPlainText())

    # Добавляем новую пару ключ-значение в словарь
    data[key] = value

    # Преобразуем словарь обратно в отформатированную JSON-строку
    # indent=4 - отступы для красивого форматирования
    # ensure_ascii=False - сохраняем кириллицу и другие символы
    self.output_text.setPlainText(json.dumps(data, indent=4, ensure_ascii=False))

    # Очищаем поле ввода ключа
    self.key_input.clear()

    # Очищаем поле ввода значения
    self.value_input.clear()
```



```
def create_json(self):
    """Создание нового пустого JSON-документа"""

    # Устанавливаем в поле вывода пустой JSON-объект
    self.output_text.setPlainText("{}")

    # Сбрасываем текущий файл (так как создаем новый)
    self.current_file = None

    # Показываем сообщение об успешном создании
    QMessageBox.information(self, "Успех", "Создан новый пустой JSON!")

def save_to_file(self):
    """Сохранение текущего JSON в файл"""

    # Получаем текст из поля вывода
    data = self.output_text.toPlainText()

    # Проверяем, есть ли данные для сохранения
    if not data:
        # Показываем предупреждение, если данных нет
        QMessageBox.warning(self, "Ошибка", "Нет данных для сохранения!")
        # Выходим из функции
        return

    # Проверяем валидность JSON (вызовет исключение, если JSON невалиден)
    json.loads(data)

    # Открываем диалоговое окно для выбора места сохранения
    # Параметры:
    # self - родительское окно
    # "Сохранить JSON файл" - заголовок окна
    # "" - начальная директория (по умолчанию)
    # "JSON Files (*.json);;All Files (*)" - фильтры файлов
    file_name, _ = QFileDialog.getSaveFileName(
        self, "Сохранить JSON файл", "", "JSON Files (*.json);;All Files (*)"
    )

    # Если пользователь выбрал файл (не нажал "Отмена")
    if file_name:
        # Проверяем, есть ли у файла расширение .json
        if not file_name.lower().endswith('.json'):
            # Добавляем расширение, если его нет
            file_name += '.json'

        # Открываем файл для записи в кодировке UTF-8
        with open(file_name, 'w', encoding='utf-8') as f:
            # Записываем JSON-данные в файл
            f.write(data)

        # Сохраняем путь к текущему файлу
        self.current_file = file_name

        # Показываем сообщение об успешном сохранении
        QMessageBox.information(self, "Успех", f"Файл сохранен: {file_name}")
```

```
def load_from_file(self):
    """Загрузка JSON из файла"""

    # Открываем диалоговое окно для выбора файла
    file_name, _ = QFileDialog.getOpenFileName(
        self, "Открыть JSON файл", "", "JSON Files (*.json);;All Files (*)"
    )

    # Если пользователь выбрал файл
    if file_name:
        # Открываем файл для чтения в кодировке UTF-8
        with open(file_name, 'r', encoding='utf-8') as f:
            # Загружаем JSON из файла в словарь Python
            data = json.load(f)

        # Преобразуем словарь в отформатированную JSON-строку и выводим
        self.output_text.setPlainText(json.dumps(data, indent=4, ensure_ascii=False))

        # Сохраняем путь к текущему файлу
        self.current_file = file_name

        # Показываем сообщение об успешной загрузке
        QMessageBox.information(self, "Успех", f"Файл загружен: {file_name}")

def clear_all(self):
    """Очистка всех полей и сброс состояния"""

    # Очищаем поле вывода JSON
    self.output_text.clear()

    # Очищаем поле ввода ключа
    self.key_input.clear()

    # Очищаем поле ввода значения
    self.value_input.clear()

    # Сбрасываем текущий файл
    self.current_file = None

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = JSONEditor()
    window.show()
    sys.exit(app.exec_())
```